

What is Linux Administration?

Linux administration is about setting up disaster recovery, managing new system builds, creating a backup to restore data, Linux hardware management, managing storage, handling file systems, and managing the security of Linux systems. A big part of Linux administration is ensuring that Linux powered systems are stable and secure.

What Should a Linux Administrator Should Know?

Typically Linux system administrators are expected to handle Linux file systems, manage the root user, have a working knowledge of bash commands, and an ability to manage users.

What Are The Duties Of System Administrators In Linux?

Commonly, the Linux **administration role** typically involves:

1. Maintenance of a Linux environment.
2. Troubleshooting and providing support when there's an issue with Linux servers.
3. Analysis of log files(mainly error logs).
4. Support of LAN and web applications.
5. Creation of operational and project-specific solutions for the organization
6. Ability to proactively figure out ways to enforce strong security practices, and increase scalability of your Linux environment

The **root user**, also known as the **superuser** or **administrator**, is a special user account in Linux used for system administration. It is the most privileged user on the Linux system and it has access to all commands and files.

Steps to login as superuser:

In Ubuntu, the superuser is known as "root". You can log in as root using the following steps:

1. Open a terminal window.
2. Type the following command to switch to the root user:

```
sudo su
```

This will prompt you for your user password. After entering your password, you will be logged in as the root user.

passwd command

It enables users to change their password on a Linux system. Regular users on Linux can only change their password. But the root user (Super User) can change any user account password in Linux and there are no restrictions.

Type following passwd command to change your **own password**:

```
$ passwd
```

Sample Outputs:

(current) Linux password:

New Linux password:

Retype new Linux password:

passwd: password updated successfully

1. A normal user may only change the password for his/her own account.
2. The **superuser (root user)** may change the password for **any account or specific account**.
3. The passwd command also changes the account or associated password validity period.

To change other user password first, login as the root user. Use `sudo -s` or `su -` command to login as root.

To change password of specific user account, use the following syntax:

```
passwd userNameHere
```

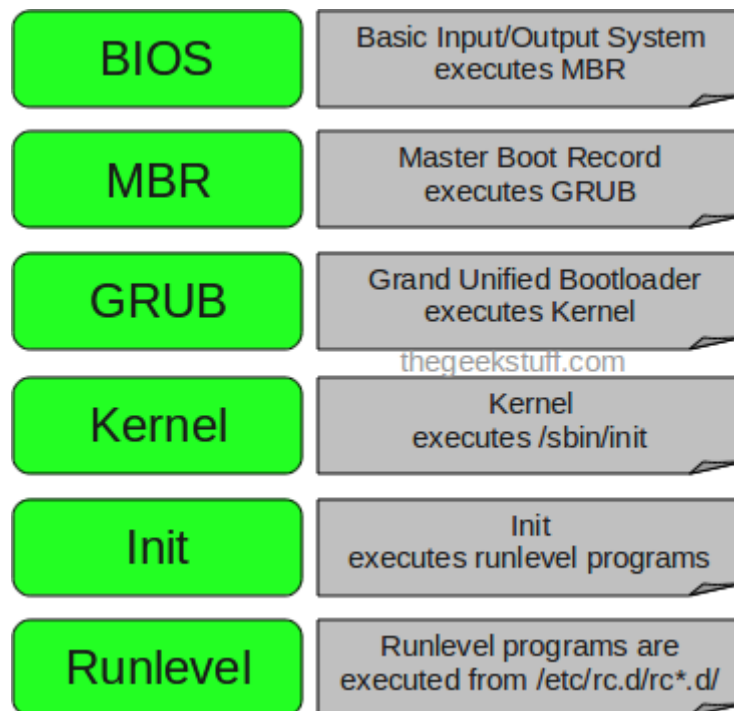
```
root@wks01:~# passwd vivek
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
```

To force a user to change his/her password, first of all the password must have expired and to cause a user's password to expire, you can use the **passwd command**, which is used to change a user's password by specifying the `-e` or `--expire` switch along with username as shown.

```
# passwd --expire ravi
```

Linux Start up Process:

The following are the 6 high level stages of a typical Linux boot process (Startup process).



BIOS

- BIOS stands for Basic Input/Output System
- Performs some system integrity checks
- Searches, loads, and executes the boot loader program.
- It looks for boot loader in floppy, cd-rom, or hard drive. You can press a key (typically F12 or F2, but it depends on your system) during the BIOS startup to change the boot sequence.
- Once the boot loader program is detected and loaded into the memory, BIOS gives the control to it.
- So, in simple terms BIOS loads and executes the MBR boot loader.

2. MBR

- MBR stands for Master Boot Record.
- It is located in the 1st sector of the bootable disk. Typically /dev/hda, or /dev/sda

- MBR is less than 512 bytes in size. This has three components

- 1) primary boot loader info in 1st 446 bytes
- 2) partition table info in next 64 bytes
- 3) mbr validation check in last 2 bytes.

- It contains information about GRUB (or LILO in old systems).
- **So, in simple terms MBR loads and executes the GRUB boot loader.**

3. GRUB

- GRUB stands for Grand Unified Bootloader.
- If you have multiple kernel images installed on your system, you can choose which one to be executed.
- GRUB displays a splash screen, waits for few seconds, if you don't enter anything, it loads the default kernel image as specified in the grub configuration file.
- GRUB has the knowledge of the filesystem
- Grub configuration file is /boot/grub/grub.conf (/etc/grub.conf is a link to this).
- **So, in simple terms GRUB just loads and executes Kernel and initrd images.**

4. Kernel

- Mounts the root file system as specified in the "root=" in grub.conf
- Kernel executes the /sbin/init program
- Since init was the 1st program to be executed by Linux Kernel, it has the process id (PID) of 1. Do a "ps -ef | grep init" and check the pid.
- initrd stands for Initial RAM Disk.
- initrd is used by kernel as temporary root file system until kernel is booted and the real root file system is mounted. It also contains necessary drivers compiled inside, which helps it to access the hard drive partitions, and other hardware.

5. Init

1. Looks at the /etc/inittab file to decide the Linux run level.
2. Following are the available run levels
 - 0 – halt
 - 1 – Single user mode
 - 2 – Multiuser, without NFS
 - 3 – Full multiuser mode
 - 4 – unused
 - 5 – X11
 - 6 – reboot
3. Init identifies the default initlevel from /etc/inittab and uses that to load all appropriate program.
4. Execute „grep initdefault /etc/inittab“ on your system to identify the default run level
5. Typically you would set the default run level to either 3 or 5.

6. Runlevel programs

- When the Linux system is booting up, you might see various services getting started. For example, it might say "starting sendmail OK". Those are the runlevel programs, executed from the run level directory as defined by your run level.
- Depending on your default init level setting, the system will execute the programs from one of the following directories.
- Run level 0 – /etc/rc.d/rc0.d/
- Run level 1 – /etc/rc.d/rc1.d/
- Run level 2 – /etc/rc.d/rc2.d/
- Run level 3 – /etc/rc.d/rc3.d/
- Run level 4 – /etc/rc.d/rc4.d/
- Run level 5 – /etc/rc.d/rc5.d/
- Run level 6 – /etc/rc.d/rc6.d/

shutdown

The **shutdown** command in Linux is used to shutdown the system in a safe way. You can shutdown the machine immediately, or schedule a shutdown using 24 hour format.

It brings the system down in a secure way.

When the shutdown is initiated, all logged-in users and processes are notified that the system is going down, and no further logins are allowed. Only root user can execute shutdown command .

Syntax of shutdown Command

shutdown [OPTIONS] [TIME] [MESSAGE]

- Shutdown options such as halt, power-off (the default option) or reboot the system.
- The time argument specifies when to perform the shutdown process.
- The message argument specifies a message which will be broadcast to all users.

Options:

- r : Requests that the system be rebooted after it has been brought down.
- h : Requests that the system be either halted or powered off after it has been brought down, with the choice as to which left up to the system.
- H : Requests that the system be halted after it has been brought down.
- P : Requests that the system be powered off after it has been brought down.

In it's simplest form when used without any argument, shutdown will power off the machine.

\$sudo shutdown

The following example will schedule a system shutdown at 05 A.M:

\$sudo shutdown 05:00

The following example will schedule a system shutdown in 20 minutes from now:

\$sudo shutdown +20

How to shutdown the system immediately To shutdown your system immediately you can use +0 or its alias now:

\$sudo shutdown now

The following command will shut down the system in 10 minutes from now and notify the users with message "System upgrade":

\$sudo shutdown +10 "System upgrade"

It is important to mention that when specifying a custom wall message you must specify a time argument too.

How to make shutdown power-off machine Although this is by default, you can still use the -P option to explicitly specify that you want shutdown to power off the system.

\$shutdown -P

How to reboot using shutdown For reboot, the option is -r.

\$shutdown -r

You can also specify a time argument and a custom message:

\$shutdown -r +5 "Updating Your System"

The command above will reboot the system after 5 minutes and broadcast Updating Your System"

How to cancel a scheduled shutdown If you have scheduled a shutdown and you want to cancel it you can use the -c argument:

sudo shutdown -c

When canceling a scheduled shutdown, you cannot specify a time argument, but you can still broadcast a message that will be sent to all users.

\$sudo shutdown -c "Canceling the reboot"

Run Levels in Linux

A run level is a state of **init** and the whole system that defines what system services are operating. Run levels are identified by numbers.

Whenever a LINUX system boots, firstly the **init** process is started which is actually responsible for running other start scripts which mainly involves initialization of you hardware, bringing up the network, starting the graphical interface.

- Now, the **init** first finds the default **runlevel** of the system so that it could run the start scripts corresponding to the default run level.
- A **runlevel** can simply be thought of as the state your system enters like if a system is in a single-user mode it will have a **runlevel 1** while if the system is in a multi-user mode it will have a **runlevel 5**.

runlevel in other words can be defined as a **preset single digit integer** for defining the operating state of your LINUX or UNIX-based operating system. Each runlevel designates a different system configuration and allows access to different combination of **processes**.

The standard **LINUX kernel** supports these seven different runlevels :

Run Level	Mode	Action
0	Halt	Shuts down system
1	Single-User Mode	Does not configure network interfaces, start daemons, or allow non-root logins
2	Multi-User Mode	Does not configure network interfaces or start daemons.
3	Multi-User Mode with Networking	Starts the system normally.
4	Undefined	Not used/User-definable
5	X11	As runlevel 3 + display manager(X)
6	Reboot	Reboots the system

By default most of the LINUX based system boots to runlevel 3 or runlevel 5. start scripts for each run level will be different performing different tasks. These start scripts corresponding to each run level can be found in special files present under **rc sub directories**.

At **/etc/rc.d** directory there will be either a set of files named **rc.0, rc.1, rc.2, rc.3, rc.4, rc.5** and **rc.6**, or a set of directories named **rc0.d, rc1.d, rc2.d, rc3.d, rc4.d, rc5.d** and **rc6.d**.

Changing runlevel

init is the program responsible for altering the run level which can be called using **telinit** command.

NOTE : The changing of runlevels is a task for the super user and not the normal user that's why it is necessary to be logged in as super user for the successful execution of the above telinit command or you can use **sudo** command as :

// using sudo to execute telinit

/*using telinit to change runlevel from 3 to 5*/

sudo telinit 5

To check Current Run Level # who -r or # runlevel

On most Linux server system default run level is 3 and on most Linux Desktop system default run level is 5.

cron is a utility in Unix-like operating systems that allows users to schedule and automate the execution of commands or scripts at specified intervals or on a recurring basis. With **cron**, users can define "crontab" files that contain commands or scripts and specify when and how often those commands or scripts should be run.

Here's an overview of how **cron** works:

1. Users create a crontab file using the **crontab** command. This file contains one or more entries, each of which specifies a command or script to run and when to run it.
2. Each entry consists of five fields, separated by spaces, that define the time and date when the command should be executed. The fields specify the minute, hour, day of the month, month, and day of the week, respectively. Users can use various symbols to specify more complex schedules, such as wildcards (*) to match any value, ranges (-) to specify a range of values, and steps (/) to specify intervals.
3. When **cron** is started, it reads the crontab files for each user and schedules the commands or scripts to be executed based on the specified intervals.
4. When a scheduled time arrives, **cron** executes the corresponding command or script in a separate process.

cron is a powerful tool that can be used for a variety of tasks, such as regularly backing up data, running system maintenance tasks, and scheduling periodic reports or updates.

Adding new cron job:

1. Open a terminal window on your Unix-like system.
2. Type the command **crontab -e** and press Enter. This will open the default crontab file for the current user in the system's default text editor.
3. Add a new line to the file that specifies the command or script you want to run and when to run it. For example, the following line will run a backup script every day at midnight:

```
0 0 * * * /home/user/scripts/backup.sh
```

This line specifies that the script **/home/user/scripts/backup.sh** should be run at minute 0 and hour 0 (midnight) every day of the month, every month of the year, and every day of the week. The fields are separated by spaces and are in the order minute, hour, day of the month, month, day of the week.

Design a shell script that takes backup of Desktop folder everyday.

```
#!/bin/bash

# Set the backup destination directory
backup_dir=/home/user/backup

# Set the source directory to be backed up
source_dir=/home/user/Desktop

# Set the backup filename with current date
backup_filename="backup-$(date +%Y-%m-%d-%H-%M-%S).tar.gz"

# Create the backup archive using tar
tar -czf "$backup_dir/$backup_filename" "$source_dir"
```

Here's how the script works:

1. The script sets the `backup_dir` variable to the directory where backups should be stored. You can change this to a directory of your choice.
2. The `source_dir` variable is set to the directory you want to back up. In this example, it's set to `/home/user/Desktop`, but you can change this to any directory you want to back up.
3. The script sets the `backup_filename` variable to a string that includes the current date and time, so that each backup has a unique filename.
4. Finally, the `tar` command is used to create a compressed archive of the `source_dir` directory and save it to the `backup_dir` directory with the `backup_filename`.

User management and group management

User management involves creating, modifying, and deleting user accounts on a system. This includes setting the user's password, home directory, and shell, as well as assigning them to one or more groups. In Ubuntu, user management is typically done using the **useradd**, **usermod**, and **userdel** commands.

Group management involves creating, modifying, and deleting groups on a system. Each user on a Linux system is assigned to one or more groups, which define their access permissions to files and directories. In Ubuntu, group management is typically done using the **groupadd**, **groupmod**, and **groupdel** commands.

Some common tasks involved in user and group management in Ubuntu include:

1. Creating a new user account: This involves using the **useradd** command to create a new user, and setting their password, home directory, and shell.
2. Modifying a user account: This involves using the **usermod** command to modify an existing user's properties, such as their password, home directory, or group membership.
3. Deleting a user account: This involves using the **userdel** command to delete an existing user account from the system.
4. Creating a new group: This involves using the **groupadd** command to create a new group on the system.
5. Modifying a group: This involves using the **groupmod** command to modify an existing group's properties, such as its name or group ID.
6. Deleting a group: This involves using the **groupdel** command to delete an existing group from the system.

Overall, effective user and group management is essential for ensuring the security and stability of a Ubuntu system, and for providing users with the appropriate access permissions to perform their tasks.

Examples:

User Management:

1. Create a new user named "jdoe": **sudo adduser jdoe**
2. Change the password for the user "jdoe": **sudo passwd jdoe**

3. Delete a user named "jsmith": **sudo deluser jsmith**
4. Lock the password for a user named "jsmith": **sudo passwd -l jsmith**
5. Unlock the password for a user named "jsmith": **sudo passwd -u jsmith**
6. Change the home directory for a user named "jdoe": **sudo usermod -d /new/home/directory jdoe**
7. Set the default shell for a user named "jdoe" to bash: **sudo usermod -s /bin/bash jdoe**
8. Force a user named "jdoe" to change their password on next login: **sudo passwd -e jdoe**

Group Management:

1. Create a new group named "developers": **sudo addgroup developers**
2. Add a user named "jdoe" to the "developers" group: **sudo adduser jdoe developers**
3. Remove a user named "jdoe" from the "developers" group: **sudo deluser jdoe developers**
4. Add a user named "jsmith" to the "sudo" group: **sudo adduser jsmith sudo**
5. List the members of a group named "developers": **getent group developers**
6. Delete a group named "testgroup": **sudo delgroup testgroup**

User configuration and password files

There are four main user administration files –

- **/etc/passwd** – Keeps the user account and password information. This file holds the majority of information about accounts on the Unix system.
- **/etc/shadow** – Holds the encrypted password of the corresponding account. Not all the systems support this file.
- **/etc/group** – This file contains the group information for each account.
- **/etc/gshadow** – This file contains secure group account information.

Maintaining Security

Linux is the most commonly used operating system for web-facing computers, running on nearly 75% of servers. Following are a few basic Linux hardening and Linux server security best practices

Use Strong and Unique Passwords. ...

2. Generate an SSH Key Pair. ...

3. Update Your Software Regularly. ...

4. Enable Automatic Updates. ...

5. Avoid Unnecessary Software. ...

6. Disable Booting from External Devices. ...

7. Close Hidden Open Ports. ...

8. Scan Log Files with Fail2ban.

9. Utilize Backups and Test Them Often

10. Perform Security Audits